

通过保护性重路由提升网络可用性

David Wetherall, Abdul Kabbani*, Van Jacobson, Jim Winget, Yuchung Cheng,
Charles B. Morrey III, Uma Moravapalle, Phillipa Gill, Steven Knight, Amin Vahdat
Google

ACM SIGCOMM 2023 · 经验报告赛道

译本说明：本译本依据用户提供的 3603269.3604867.pdf（原刊第 684–695 页）逐段翻译，由当前对话助手直接理解并撰写。正文、摘要、图注、脚注与致谢均译为中文；人名、机构名、协议名及参考文献书目信息保留原文。原图保留矢量内容，其英文坐标、图例和标注在图下给出中文对照。原文的数字或公式疑点照录并另作译者注。单栏重排后的页码与原文不同。

摘要

原文位置：PDF 第 1 页 / 原刊第 684 页。

我们提出 PRR (Protective ReRoute, 保护性重路由)，这是一种传输层技术，用来缩短用户可感知的中断时间，并与路由修复形成互补。它可以加入任何传输协议，使其在多路径网络中受益。PRR 响应流连通性失效信号（例如重传超时），改变该流分组中的 FlowLabel，从而使交换机和主机选择一条可能绕开中断的不同网络路径。为支持这一机制，我们调整了 IPv6 网络架构，使其使用 FlowLabel，这样主机就能在无需应用参与的情况下改变其流的路径。PRR 已在 Google 全网部署，用于 TCP 和 Pony Express，并已保护全部生产流量数年之久。我们的 Cloud 客户也可以使用它。我们发现，PRR 对实际中断非常有效。在对骨干网进行的一项测量研究中，加入 PRR 后，RPC 流量的区域对累计中断时长减少了 63–84%。这相当于为可用性增加了 0.4–0.8 个“9”。

CCS 分类 网络 → 数据路径算法；网络可靠性；端节点。

关键词 网络可用性，多路径，FlowLabel。

ACM 引用格式 David Wetherall, Abdul Kabbani, Van Jacobson, Jim Winget, Yuchung Cheng, Charles B. Morrey III, Uma Moravapalle, Phillipa Gill, Steven Knight, and Amin Vahdat. 2023. Improving Network Availability with Protective ReRoute. In *ACM SIGCOMM 2023 Conference (ACM SIGCOMM '23)*, September 10, 2023, New York, NY, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3603269.3604867>

原文版权说明 允许为个人或课堂用途免费制作本作品全部或部分内容的数字副本或纸质副本，前提是副本的制作或分发不以营利或商业利益为目的，且在首页附有本声明和完整引用信息。本作品中第三方组成部分的版权必须得到尊重。其他用途请联系版权人或作者。ACM SIGCOMM '23, 2023 年 9 月 10 日, New York, NY, USA. © 2023, 版权由版权人或作者持有。ACM ISBN 979-8-4007-0236-5/23/09. <https://doi.org/10.1145/3603269.3604867>

*该作者在 Google 任职期间参与了本项工作。

1 引言

原文位置: PDF 第 1-2 页 / 原刊第 684-685 页。

超大规模服务提供商运营着支撑数十亿客户服务的网络，其使用场景涵盖银行、医院和商业活动。网络的高可用性至关重要。由于任何大型网络都不可能完全避免故障，因此必须快速修复故障，以免造成服务中断。经典修复策略依赖路由协议，将流量迅速从失效的链路和交换机转移到可用路径上。

根据我们的经验，大型网络实现高可用性的主要障碍，是有相当数量的故障无法通过路由迅速恢复连通性。有时，配置错误、软件缺陷，或只有在硬件故障等诱发事件发生后才暴露的意外交互，会使路由无法发挥作用。例如，交换机中的缺陷可能导致分组被丢弃，却没有同时将相应端口声明为不可用；交换机也可能与其 SDN 控制器失去联系，使部分网络无法被控制；路由或流量工程还可能使用错误的权重，导致链路过载。

尽管这些复杂故障发生的概率不高，但由于规模庞大，它们确实会出现在超大规模网络中。这些故障需要更高层的修复工作流介入，因此会造成持续时间较长的中断，进而对可用性产生不成比例的影响：仅一次 5 分钟的中断，就会使月度正常运行时间比例低于 99.99%。从用户感受来看，持续数分钟的中断会造成严重干扰，而仅持续数秒的短暂中断可能根本不会被察觉。

或许令人意外的是，即使面对路由能够修复的中断，仅靠路由也不够。流量控制系统工作在多个时间尺度上。快速重路由 [3,4] 能在数秒内执行局部修复，在单点故障发生时将流量转移到备用路径。路由执行全局修复；在超大型网络中，传播拓扑更新、计算新路由并将其安装到交换机，可能需要数十秒。最后，流量工程在数分钟内作出响应，通过调整路径权重，使需求与容量相匹配。

要仅靠路由维持高可用性，就需要快速重路由每次都成功。然而，有相当一部分中断只有通过全局路由和流量工程才能完全恢复。这是因为快速重路由依赖有限的备用路径。这些路径是针对预先设想的故障所对应的共享风险链路组 (SRLG) [9] 预计算的，因此可能无法覆盖实际故障。即便备用路径在故障中幸存下来，其性能也不如它们所替代的失效路径。路径数量随路径长度呈指数增长，因此，与全局范围内绕开同一故障的端到端路径相比，故障相邻交换机之间可选择的局部路径更少。在可选路径有限、故障又造成容量损失的情况下，备用路径很容易过载。

我们的可靠性架构把快速修复的责任从路由转移给主机。现代网络为此提供了机会，因为网络的扩展方式已经不仅是提高链路速率，而是增加更多链路。要增加容量，这些链路就必须彼此分离。于是，一对端点之间会存在多条能够独立失效的路径。因此，扩展容量所采用的策略，同时也增加了能够提升可靠性的路径多样性。

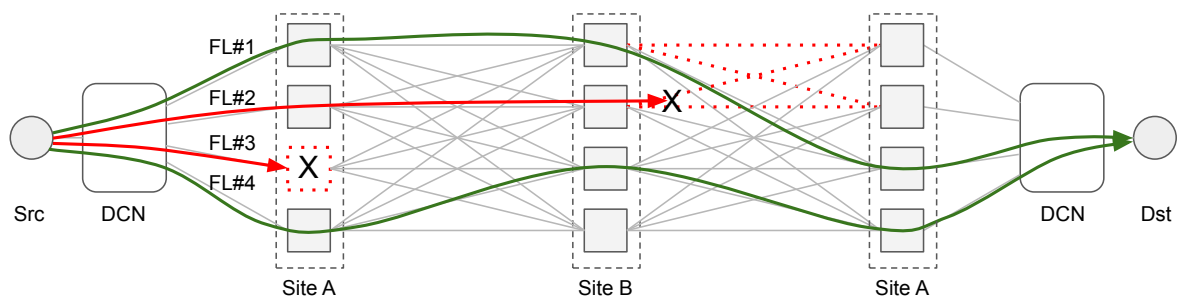


图 1: 主机之间的多条路径能够绕开图中红色虚线所示的故障。

图中文字对照: Site A / Site B: 站点 A / 站点 B; DCN: 数据中心网络; Src / Dst: 源端 / 目的端; FL#1-FL#4: 不同的 FlowLabel 取值; X: 故障位置。

主机通常观察到双峰式的中断行为：到达某一目的端的部分连接所经过的路径是会在网络中丢弃分组的黑洞，而到达同一目的端的其他连接所经过的路径仍然正常工作。这种部分连通性丧失，是多路径与避免单点故障的标准技术共同作用的结果。例如，一个故障不太可能同时影响地理位置不同的光纤线路，也不太可能影响某个为提高可靠性而分片的网络实体的全部分片。

为了恢复连通性，主机必须避开失效路径并使用可用路径。但这带来了一个难题：IPv4 架构不允许主机影响一条流在网络中的路径。相反，随着多路径的兴起，交换机上的等价多路径（ECMP）路由 [24, 48] 使用传输层标识符将不同流分散到各条路径上。这种设计将连接与路径绑定在一起。幸运的是，如今我们已经将 Google 网络迁移到 IPv6，可以按照 IPv6 FlowLabel [2, 34] 在架构中的原定作用，用它来影响连接的路径。

采用保护性重路由（PRR）后，交换机将 FlowLabel 纳入 ECMP 哈希。主机随后只需改变分组中的 FlowLabel，就能为连接重新选路。基于主机在传输层失效（例如 TCP 超时）时执行重路由这一由来已久的思想，Linux 已经为 IPv6 支持了这种行为。我们通过处理确认分组遭遇的中断，补全了这项支持。PRR 能在 RTT 时间尺度内修复失效路径，而不引入额外开销。它适用于所有可靠传输协议，包括多路径传输协议（第 2.5 节）。它的实现规模很小，而且可以在主机和交换机上逐步部署。

过去数年，PRR 一直在 Google 全球网络中全天候运行，保护全部 TCP 和 Pony Express [31] 流量；Pony Express 是一种绕过操作系统的数据中心传输协议。最近，我们又将其扩展到 Google Cloud 流量。由于厂商支持 IPv6 FlowLabel 哈希需要交付周期，加上内核和交换机升级，这一转变耗时较长。但可用性的改善非常值得：在一项持续 6 个月的研究中，对骨干网的全网监测发现，PRR 使 RPC 流量的区域对累计中断时长（即网络中所有区域对之间中断时间的总和）减少了 63–84%。

本工作有两项贡献。第一，我们描述大型网络中中断的特征，通过案例研究强调那些损害可用性的长时间中断。第二，我们给出 PRR 在全网范围内的效果数据，以及从生产实践中获得的经验。我们希望这项工作能鼓励其他人利用 IPv6 FlowLabel，采用这样的架构：路由为主机提供多条多样化路径，而传输协议通过转移流来提升可靠性。

本工作不涉及伦理问题。

2 设计

原文位置：PDF 第 2–5 页／原刊第 685–688 页。

下面介绍 PRR：先说明我们的网络架构，再介绍在该架构中如何检测中断和重新选路。

2.1 多路径架构

PRR 面向这样一类网络：路由为每对主机提供多条路径；大多数现代网络都是如此。例如，图 1 展示了一个抽象的广域网，其中一对主机同时使用了众多可能路径中的四条。发生故障时，部分路径会失去连通性，但其他路径可能仍然可用。图中，一条路径因交换机故障而失效，另一条因链路故障而失效；另外两条路径则没有受到这些故障的影响。

多路径为主机提供了替用户修复故障影响的机会。为此，需要让主机能够控制其连接的路径选择。我们调整了 IPv6 网络架构，通过使用 FlowLabel [2] 来实现这一目标。

在最初的 IPv4 互联网中，设计者有意让主机不知道端到端流所经过的路径。端主机与路由之间的这种分离，适合依靠提高每条链路速率来扩展规模的网络，因为这类网络中主机之间的 IP 层路径相对较少。过去二十年里，网络越来越多地通过并行增加链路来扩展容量，从而产生了大量

额外的可行路径。要充分利用容量，必须把流分散到这些路径上，因此基于 ECMP 的多路径技术 [24, 48] 已成为网络的核心组成部分。这些为利用容量、改善性能而承载不同流的路径，也增加了有利于可靠性的多样性。

IPv6 架构允许主机通过 FlowLabel 直接管理多条路径：主机使用它告诉交换机，哪些分组需要走同一条路径，而无需知道该路径的具体细节 [34]。这种安排使传输协议能够让流沿单条路径传输，同时允许 IP 网络通过并行链路扩展规模。

然而，IPv6 直到 2015 年前后才得到广泛部署，上述用法并未流行起来。在此期间，ECMP 使用传输层首部实现多路径。例如，每台交换机对各个分组的 IP 地址和 TCP / UDP 端口进行哈希，从可用的下一跳中伪随机地选出一个。这种做法缓解了扩展问题，却把每条 TCP 连接限制在一条固定的网络路径上。

IPv6 的采用使我们能够按原定用途使用 FlowLabel。我们在交换机上把 FlowLabel 纳入 ECMP 哈希；各厂商已经普遍提供这项能力。¹作为务实的一步，我们扩展 ECMP，使其同时使用 FlowLabel 和通常使用的四元组。关键收益在于：主机无需改变传输层标识符，就能改变路径。在图 1 中，仅改变 FlowLabel，就可以让一条连接在四条路径之间切换。

2.2 算法概览

PRR 建立在主机响应传输问题而进行重路由的思想之上。主机为每条连接运行一个 PRR 实例，以保护到远端主机的正向路径。之所以需要按连接运行，是因为路由和 ECMP 会使不同连接走不同路径，所以一个 PRR 实例无法从另一个实例那里获知哪些路径可用。

当一条连接的网络路径失去 IP 连通性时，该连接就发生了中断。如果连接在中断期间处于活跃状态，PRR 会在较短时间后通过传输协议检测到中断事件，具体见第 2.3 节。中断可能来自正向路径、反向路径或双向路径失效。由于流量工程会使用非最短路径，主机之间的路由通常不对称，因此单向故障相当常见 [23]。

PRR 响应中断事件，触发旨在恢复连通性的重新选路，具体见第 2.4 节。如果新路径成功恢复了连通性，对这条连接而言，中断就结束了，尽管故障本身可能仍然存在。如果没有恢复，PRR 会在一段时间后检测到后续中断事件，并再次触发重新选路。这个过程会持续进行，直到恢复成功、中断因外部原因结束（例如控制平面修复了故障），或连接被终止。

该过程中存在一种根本性的判断歧义：仅凭一台主机，无法区分故障发生在正向、反向还是双向路径。因此，中断事件可能导致 PRR 进行不必要的重新选路，使原本没有失效的正向路径也出现失效，从而拖慢恢复。幸运的是，这并不影响正确性，因为中断事件会持续触发重新选路，直到正向和反向路径都恢复。

2.3 TCP 中断检测

所有可靠传输协议都可以实现 PRR。我们以 TCP 这个重要实例来说明其工作方式。由于网络中断并不常见，PRR 在没有中断时的开销必须可接受。这意味着，在主机状态、处理操作和报文方面，PRR 都必须非常轻量。我们的做法是把中断检测纳入 TCP 的正常处理过程。

数据路径。如何推断中断有许多选择，从分组丢失到连续超时都可以考虑。对于已经建立的连接，PRR 将 Google 网络中每次 TCP RTO（重传超时）[35] 都视为一个中断事件。当连通性受损到连接无法继续取得进展时，这种方法会产生一个按照指数退避间隔反复出现的信号。RTO 可能是

¹在我们开始时，相关支持还很有限，但此后一直在稳步增加。

误触发的，也可能表示远端主机故障，但在这些情况下重新选路并无害处：它要么很可能成功，要么由于故障与网络无关而不起作用。

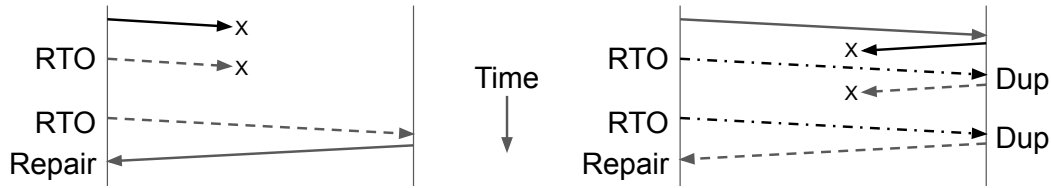


图 2：单向正向故障（左）和反向故障（右）的恢复示例。非实线表示 FlowLabel 已改变。

图中文字对照：Time：时间；RTO：重传超时；Repair：恢复；Dup：重复数据接收；X：分组丢失。

ACK 路径。有意思的是，仅靠 RTO 并不足以检测反向路径中断。ACK 路径失效不会使 ACK 发送方发生 RTO，因为 ACK 本身并不通过可靠确认来保障交付。考虑请求／响应流量：只有完整接收到请求后才会发送响应；如果一个长请求对应的纯 ACK 丢失，请求就无法被完整接收。

我们从第二次接收到重复数据开始，将重复数据接收视为 ACK 路径已经失效的信号。一次重复通常是由误重传或尾部丢失探测（TLP）[10]引起的，而第二次重复则很可能表示 ACK 丢失。

控制路径。最后，在建立新连接时，我们采用类似的方法检测中断。在客户端到服务端的方向上使用 SYN 超时，在服务端到客户端的方向上使用接收到 SYN 重传这一信号。

性能。PRR 的性能取决于 RTO 驱动这些事件序列的速度。在 Google 之外，对于已经建立的连接，第一次 RTO 的一个合理经验近似是

$$RTO = SRTT + RTTVAR \approx 3 RTT,$$

且最小值为 200 ms。在 Google 内部，我们使用默认的 Linux TCP RTO 公式 [36]，但将 RTTVAR 的下限和最大延迟 ACK 时间，分别从默认的 200 ms 和 40 ms 降至 5 ms 和 4 ms [8]。因此，一个合理的经验近似是

$$RTO \approx RTT + 5 \text{ ms}.$$

这使都市圈内的 RTO 可以低至个位数毫秒，同一洲内为数十毫秒，更长路径则为数百毫秒。与外部网络的上述经验近似相比，更低的 RTO 使 PRR 加快了 3–40 倍。对于新连接，第一次 SYN 超时通常为 1 s，处于 RTO 取值的较高一端。这意味着，在中断期间建立连接，要比修复已有连接耗费明显更长的时间。

示例。图 2 展示了请求／响应流量遭遇单向路径故障时的恢复过程。为简化说明，图中重点画出了初次发送以及后来涉及重新选路的发送，省略了快速恢复、TLP 等机制中的其他分组。每条非实线都表示 FlowLabel 已改变。

正向故障的行为很简单：每次 RTO 都会触发带有重新选路的重传（虚线）。这一过程持续到找到一条可用的正向路径为止。由于反向路径正常，随后即可直接恢复。重新选路的次数（本例为两次）是随机的，其期望值取决于可用路径所占的比例。

反向故障有两点不同。由于正向路径原本正常，RTO 会导致不必要的重新选路（点划线）。但因为只有反向路径存在故障，这种操作并无害处。远端主机检测到重复数据后（在图中未画出的 TLP 之后），会在反向路径上重新选路，直到找到可用路径。尽管存在这些差异，恢复时间与正向故障的情况相同。

双向故障更复杂，因为修复行为取决于连接最初是在正向路径、反向路径，还是两个方向同时失效。与单向故障不同，此时可能出现有害的不必要重新选路，以及反向重新选路被延迟的情况。

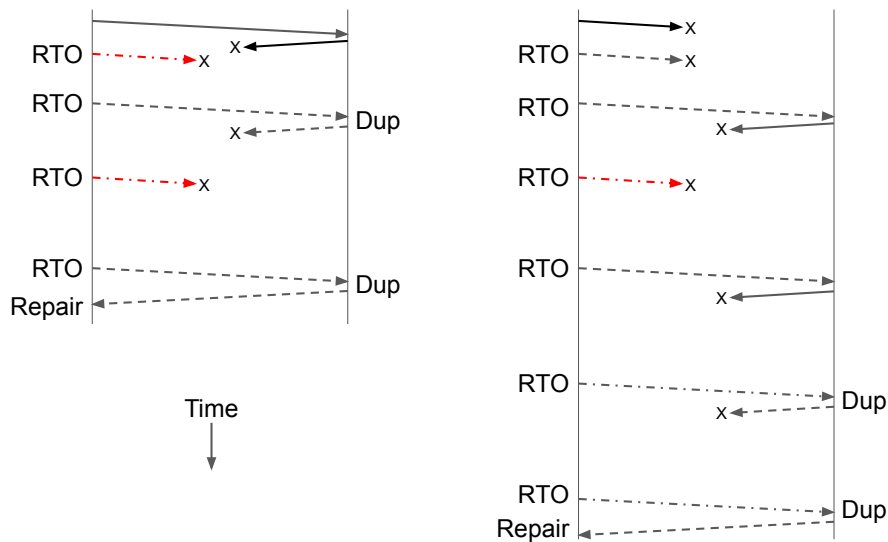


图 3: 双向故障的恢复示例: 连接最初仅反向路径失效 (左), 或两个方向同时失效 (右)。非实线表示 FlowLabel 已改变; 红色点划线表示有害的重新选路。

图中文字对照: Time: 时间; RTO: 重传超时; Repair: 恢复; Dup: 重复数据接收; X: 分组丢失。

如果连接最初遇到正向路径失效, 但碰巧没有遇到反向路径失效, 那么恢复过程与单向故障相同。正向修复成功时, 分组是第一次到达接收方, 因此接收方不会重新选路, 而是使用其正常的反向路径完成恢复。

相反, 如果连接最初只有反向路径失效 (图 3 左), 则会出现不同的行为。第一次 RTO 发生时, 会进行不必要的正向重新选路。这时该操作可能有害, 而在图示例中也确实如此 (红色点划线): 它导致正向路径丢包, 需要后续 RTO 才能修复。正向路径恢复可用后, 重复数据接收会触发反向重新选路。要恢复连通性, 必须同时选中可用的正向路径和反向路径。

当连接两个方向都失效时 (图 3 右), 恢复时间最长。恢复需要反向重新选路, 但它要等到正向路径完成两次修复后才会发生。这是因为接收方直到第二次收到重复数据才会首次重新选路, 而原始发送和 TLP 都丢失了。要完成恢复, 后续仍需正反两个方向同时修复。与前面一样, 不必要的正向重新选路可能拖慢这个过程。

2.4 利用 FlowLabel 重新选路

PRR 通过使用 FlowLabel, 以本地动作的形式完成重新选路。中断期间, 它不依赖与其他网络组件 (例如 SDN 控制器或路由系统) 的通信。

随机重新选路。对于每次中断事件, PRR 都会随机改变该连接分组中使用的 FlowLabel 值。自 2015 年起, 这已是 Linux 通过 txhash 实现的默认行为; 我们最后一项提交到上游的修改 (用于 ACK 重新选路) 于 2018 年合入。操作系统执行这一动作无需应用参与, 从而大大简化了部署。

对于设计良好的 ECMP 哈希函数, 不同输入相当于在每台交换机的可用下一跳中进行一次随机抽取。如果把一条路径定义为各台交换机所做选择的串接, 那么只要路径经过的交换机不止少数几台, 改变输入就会以很高的概率改变路径。不过, 关键问题在于新路径再次经过故障位置的概率有多高。

必须通过实测确定一次中断中失效路径的比例, 因为它取决于故障的性质。这一比例还因前缀对而异, 并随时间变化。中断比例通常较小, 因此可以快速恢复。例如, 当中断比例为 25% (即四

分之一的路径失效) 时, 一次随机抽取就有 75% 的概率成功。更一般地, 对于中断比例为 $p\%$ 的 IP 前缀对, 经过 N 次重路由尝试后, 连接仍处于中断状态的概率按 p^N 下降。

译者注: 本段保留原文的 $p\%$ 和 p^N 记法。要使概率表达式在数值上成立, p^N 中的 p 必须是 $[0, 1]$ 内的失效比例, 例如 25% 应代入 $p = 0.25$; 若 p 表示百分数数值 25, 则应写作 $(p/100)^N$ 。

避免级联故障。重新选路必须避免级联故障: 不能因为某个问题而把大量流量转移到其他位置, 造成负载集中并引发后续问题。在这方面, PRR 优于路由: 快速重路由会同时以相同方式转移许多仍在活动的连接, 而 PRR 的流量转移更加渐进、平滑。

这种转移是渐进的, 因为每条 TCP 连接都会独立响应中断, 从而将反应时刻分散在 RTO 时间尺度上。每条连接都是在 RTO 之后、已经暂时停止发送时迁移的, 随后在拥塞控制的约束下逐步提高发送速率。

这种转移是平滑的, 因为随机重新选路会按照路由权重把负载分配给可用路径。在一个 RTO 间隔内, 重新选路造成的每条可用路径的期望负载增幅, 以中断比例为上界。例如, 中断比例为 50% 时, 期望负载增幅为 50%: 一半的连接重新选路, 其中一半 (即全部连接的四分之一) 落到仍然可用的另一半路径上。负载至多增长至原来的 2 倍, 通常明显低于这一水平; 这不比 TCP 慢启动 [25] 更激进, 也完全处于拥塞控制的适应范围内。此外, PRR 只能使用符合策略的路径, 这些路径具有可接受的时延等属性, 而绕行路由并没有这样的保证。

另一个相关顾虑是, 中断触发的重新选路, 可能在中断结束后仍使流量集中在网络的一部分。不过, 实践中似乎并非如此: 路由更新通过随机化 ECMP 哈希映射来分散流量, 连接的新建与终止也会纠正不均衡。

2.5 其他方案

多路径传输协议。另一种方法是使用 MPTCP [6] 或 SRD [38] 等多路径传输协议, 因为同时使用多条路径的连接更有可能在中断中继续工作; 这些协议还能够提高数据中心中的性能 [33]。但 PRR 可以应用于任何传输协议, 包括多路径传输协议, 以进一步提高可靠性。例如, MPTCP 的所有路径也可能碰巧全部失效; 而且它在建立连接时较为脆弱, 因为只有三次握手成功后才会添加子流。PRR 可以保护这些情况。

此外, 许多连接的使用负载很轻, 因此维护多条路径所需的资源与复杂性成本, 并没有相应的性能收益。把数据中心内所有使用场景都迁移到一种新传输协议的可行性, 也是我们考虑的因素。因此, 我们选择把 PRR 应用于 TCP 和 Pony Express, 以提高现有传输协议的可靠性。

应用层恢复。即使没有 IPv6 或 FlowLabel, 应用也能通过在超时后重建长时间没有取得进展的 TCP 连接, 近似实现 PRR 的效果。²在采用 PRR 之前, 我们依赖的就是这种方法。但使用较短的 RPC 超时, 成本远高于 PRR, 性能也更差。这是因为建立 RPC 通道需要多个 RTT, 而且安全握手还会带来计算开销。由于依赖应用开发者实现, 这种方法的覆盖范围也更有限。将 PRR 加入 TCP 后, 可以覆盖各种应用, 包括由交换机或主机发起的 BGP、OpenFlow 等控制流量, 从而达到避免触发应用层恢复的理想效果。

PLB。最后, PRR 与保护性负载均衡 (Protective Load Balancing, PLB) [32] 密切相关。在我们的实现中, 两者被统一起来, 共用同一种重新选路机制, 但目的不同。PLB 利用拥塞信号 (来自 ECN 和网络排队时延) 重新选路, 以平衡负载; PRR 利用连通性信号 (例如超时) 重新选路, 以修复中断。这些信号可以顺利共存, 但存在一处相互影响: 中断发生时, PRR 会启动, 把流量转移到新的可用路径上。由于中断降低了容量, 随后的网络拥塞可能触发 PLB, 使流量重新选到一条失效路径上。因此, PRR 启动后, 我们会暂停 PLB, 以避免振荡和恢复时间延长。

²Linux 默认在约 15 分钟后判定 TCP 连接失败。应用的超时时间更短。

3 仿真结果

原文位置: PDF 第 5–6 页 / 原刊第 688–689 页。

我们使用一个简单模型, 预测 PRR 如何降低失效连接的比例。目的是为读者提供一个理解 PRR 预期表现的直观模型; 后面的案例研究 (第 4.2 节) 中也会出现这些效应。

我们在多种故障模型下, 对一组 2 万条长连接进行仿真, 模拟由 TCP 指数退避驱动的重选路。这种工作负载对应于我们测量连通性时使用的主动探测。故障在 $t = 0$ 时开始, 并在每条连接于 $t \geq 0$ 后首次发送时影响该连接。我们建模黑洞丢包, 忽略拥塞丢包。如果一个分组在 2 s 内没有得到确认, 就将该连接视为失效。修复行为主要受两个因素影响, 下面依次讨论: RTO (重传超时) 和失效路径所占的中断比例。

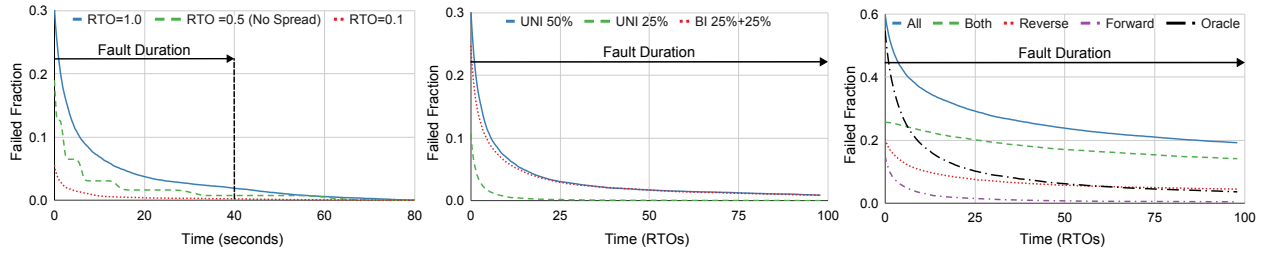


图 4: (a) RTO 的影响; (b) 单向和双向故障的修复曲线; (c) 双向故障修复过程的分解。

图中文字对照: Failed Fraction: 失效连接比例; Time (seconds): 时间 (秒); Time (RTOs): 时间 (以 RTO 为单位); Fault Duration: 故障持续时间; No Spread: 无离散分布; UNI: 单向故障; BI: 双向故障; All: 全部连接; Both: 初始双向均失效; Reverse / Forward: 初始仅反向 / 正向失效; Oracle: 理想参照。RTO 数值、百分比及曲线标记保留原图。

RTO. RTO 取决于连接的 RTT、连接状态以及 TCP 的具体实现。在我们的网络中, RTT 从都市圈内的 $O(1\text{ ms})$, 到洲内的 $O(10\text{ ms})$, 再到全球范围的 $O(100\text{ ms})$ 不等。新连接的 SYN RTO 通常为 1 s。这些差异会相应影响恢复速度。

图 4(a) 展示了三种 RTO 场景下, 单向 50% 中断 (即一半路径失效) 的修复情况。图中间的曲线对应 RTO 集中在 0.5 s 中位数附近的连接, 大部分取值介于 0.45–0.55 s。我们使用对数正态分布 $\text{LogN}(0, 0.06)$, 再以 0.5 s 的 RTO 中位数缩放, 来生成这些 RTO。连接的开始时刻还存在 1 s 的抖动。整体行为呈现“台阶”形状: 每次连接重新选路, 失效比例就下降 50%。注意, 失效比例的初始值约为 0.2, 远低于最初落入黑洞的连接比例 50%。这是因为这些黑洞连接会发生 RTO, 且大多数在 2 s 超时期限之前就已恢复。

虽然这种台阶形状便于理解, 但我们只会同质的数据子集中观察到它。更典型的是顶部和底部那样的平滑曲线, 因为不同连接在不同时间重新选路。这两条曲线的 RTO 中位数分别为 1 s 和 100 ms, 使用 $\text{LogN}(0, 0.6)$ 分布并按对应的 RTO 中位数缩放生成。该分布使 RTO 的取值更分散, 标准差是前一种情况的 10 倍。底部曲线由于 RTO 中位数更低, 修复明显更快; 同时, 由于 RTO 的分散性, 它也变得更平滑。

较小的 100 ms RTO 会产生很大影响。它既降低初始失效比例, 又使后续各次 RTO 更早到来。顶部曲线的初始 RTO 约为 1 s, 模拟了新连接以及长 RTT 情况下的失效率。由于 RTO 更大, 相应的修复也更慢。

对于这两条曲线, 失效连接比例 f 都随时间按多项式规律下降。假设中断导致比例为 p 的路径失效。经过 N 次 RTO 后, f 降至其初始值的 p^N , 即呈指数下降。然而, 各次 RTO 在时间 t 上的间隔也呈指数增长, 因此第 N 次 RTO 对应 $t \approx 2^N$ 。将这两个表达式合并, 可得

$$f \approx p^{\log_2(t)} = \frac{1}{t^K}, \quad K = -\log_p(2).$$

因此, 当 $p = \frac{1}{2}$ 时, 失效概率按 $1/t$ 下降; 当 $p = \frac{1}{4}$ 时, 则按 $1/t^2$ 下降。

译者注: 上述公式中的 $K = -\log_p(2)$ 按原文照录。由 $p^{\log_2 t} = t^{\log_2 p}$ 可知, 要与同一行等式及后面的两个例子一致, 应有 $K = -\log_2 p$ 。这应是原文对数底数与真数写反的排印疑点。本注是译者的代数核对, 并非作者发布的勘误。

一个值得注意的现象是, 故障(虚线)在 $t = 40\text{s}$ 时已经结束, 但部分连接直到 $t = 80\text{s}$ 仍未恢复连通性。也就是说, TCP 所感知的失效可能比 IP 层中断持续得更久。原因在于指数退避: 只有在故障消除后的第一次重试时, 连接才能成功交付分组并恢复。如果故障在 $t = 40\text{s}$ 时结束, 有些连接可能在区间 $[20, 40)$ 内经历过失败。这些连接会增大 RTO, 并在区间 $[40, 80)$ 内才再次重试。

中断比例。故障严重程度同样会显著影响恢复时间。图 4(b) 展示了三种长时间故障的修复过程。我们以初始 RTO 中位数为单位对时间归一化, 按照前述方式使 RTO 分散, 并把超时阈值设为 RTO 中位数的两倍。

上方实线对应单向 50% 中断, 与前述 1s RTO 曲线相对应。下方实线表示单向 25% 中断的修复情况。它具有相同的效应, 但初始失效比例更低, 下降也更快。此时, 每次 RTO 都能修复剩余连接中的 75%。

虚线表示双向中断的修复, 其中每个方向都有 25% 的路径失效。对于每条连接, 正向和反向路径独立失效, 以模拟不对称路由。这条曲线与单向 50% 中断相似, 尽管我们可能原本预期它恢复得更快, 因为同时选中可用正向和反向路径的概率是 $\frac{9}{16}$, 大于 $\frac{1}{2}$ 。原因在于, 双向中断包含三个恢复速率不同的组成部分, 下面将具体说明。

在图 4(c) 中, 我们按照第 2.3 节的说明, 将正向 50%、反向 50% 的长期中断修复曲线(实线)分解为各个组成部分(虚线)。这种中断很严峻: 75% 的往返路径已经失效, 因此在尾部阶段, 每次 RTO 只能使失效比例减少四分之一。最初只在一个方向(正向或反向)失效的连接, 恢复最快。最初两个方向都失效的连接, 则会因不必要的重新选路和反向重新选路启动较晚而缓慢恢复。为了说明这些效应的代价, Oracle 曲线(点线)展示了没有这些效应时失效比例的改善情况。

小结。我们的结论是: 对于已经建立且 RTO 较小的连接, 面对最多使一半路径成为黑洞的故障, PRR 可以在数秒内修复超过 95% 的连接。这种修复足够快, 以至于黑洞通常不会被用户察觉。对于更大的 RTO 或新连接, 要达到同样的修复程度需要数十秒, 并表现为一次短暂的服务中断。当较大的 RTO 与使绝大多数路径成为黑洞的故障同时出现时, PRR 无法避免明显的服务中断, 但仍会随着时间推移推动恢复。

4 生产环境结果

原文位置: PDF 第 6–10 页/原刊第 689–693 页。

PRR 在 Google 全网中全天候运行, 保护 TCP 和 Pony Express 流量。下面给出一项测量研究, 说明它如何为用户维持较高的网络可用性: 先讨论若干中断案例, 最后分析全网影响。

我们的研究在全球范围内观察整个生产网络中的真实中断, 覆盖 B2 和 B4 [23, 26] 两个骨干网。两者采用的技术差异很大, 从 MPLS 到 SDN, 因此故障和修复行为也不同。此外, 就我们所知, 研究结果与实际服务体验一致。这项研究的一个局限是, 我们无法提供服务体验数据, 而是报告长连接探测流的结果。尽管如此, 测量覆盖仍然十分全面, 探测次数确实达到了万亿量级。

4.1 丢失测量

我们通过主动探测监测网络中的丢失情况。具体而言，在数据中心集群之间，使用由源/目的 IP 地址及端口定义的多条流发送探测。由于 ECMP，不同流经过不同路径。每条流每分钟发送约 120 次探测，每对集群至少由 200 条流进行探测。这样的安排使我们能够同时观察丢失随时间和路径的变化，并在这两个维度上都获得较高分辨率。我们也会按网络区域对汇总测量结果；每个区域大致对应一个都市圈，包含多个集群。

我们使用三类探测，在不同网络层次观察丢失。首先，UDP 探测测量 IP 层的分组丢失，我们将其称为 L3 探测。它们用于监测底层网络的连通性，突出反映故障和路由恢复，但无法体现服务所体验到的网络表现。

为了测量采用 PRR 之前的应用性能，我们使用空的 Stubby RPC 作为探测；Stubby 是开源远程过程调用框架 gRPC [18] 的内部版本。我们将其称为 L7 探测。这些探测受益于 TCP 的可靠性和 RPC 超时机制，后者会重建无法继续取得进展的 TCP 连接。如果 RPC 未能在 2s 内完成，就视为一次 L7 探测丢失。Stubby 会在 20s 后重建 TCP 连接，与 gRPC 的默认超时一致。

最后，为测量采用 PRR 后的应用性能，我们在启用 PRR 的情况下发出 L7 探测，称为 L7/PRR 探测。它们不仅受益于 TCP 的可靠性和 RPC 超时，还受益于 PRR 的重新选路。注意，由于 PRR、TCP 和 RPC 的修复机制，底层网络可能处于中断状态，而应用却没有中断。因此，通过比较这三组探测，我们可以理解 PRR 相对于未使用它的应用以及底层网络所带来的收益。

我们的全网汇总使用了截至 2023 年年中、持续 6 个月的探测数据，覆盖核心网络中所有区域对和两个骨干网。由于并非所有探测机器都具备禁用 PRR 的功能，L7 探测的数量占比（29%）略低于 L7/PRR（37%）和 L3（33%）。但我们没有理由认为这会使结果不具代表性。

4.2 案例研究

首先，我们通过一组不同类型的重大中断案例，考察 PRR 的行为。大多数中断都比较短暂或影响范围较小。我们选择的长时间、大规模中断案例属于例外情况；之所以值得研究，是因为如果没有 PRR 或其他方法进行修复，它们会对用户造成严重干扰。

案例 1：复杂的 B4 中断。第一个案例是我们考察的持续时间最长的一次中断，历时 14 分钟，影响了通过 B4 骨干网连接的区域对。我们用它来说明在不同时间尺度上运行的多层修复机制。中断由一次罕见的双重电源故障引起：B4 的一个超级节点 [23] 中，一整机架的交换机被关闭，超级节点的其余部分也与 SDN 控制器失去联系。随后，一项无关的维护阻塞了修复工作流，使中断进一步延长。长时间中断的成因多样，行为通常很复杂。在这个案例中，单次电源故障本不会造成中断；如果修复工作流成功执行，中断也会短得多。但不幸的是，这三个事件同时发生了。

图 5 展示了中断期间的探测丢失情况。上图给出 B4 上受影响的跨洲区域对之间，L3、L7 和 L7/PRR 探测的丢失率随时间的变化；下图给出洲内区域对的相同结果。由于两者表现存在定性差异，我们将它们分开呈现。每个数据点覆盖 0.5s，因此可以细致地观察故障如何随时间破坏连通性。每个数据点都对同一时间间隔内发送的数千条流取平均，这些流实际经过了数千条路径。

先看表示 IP 层连通性的 L3 曲线。由于故障使 SDN 控制平面失去连接，它无法通过下发修复配置来推动快速恢复。约 100s 时，全局路由系统介入，对源端和目的端都不在故障邻近区域的流量重新选路。该动作降低了中断的严重程度，但没有彻底解决问题。十多分钟之后，排空工作流将故障部分从网络服务中移除，才完成修复。

整个中断期间，丢失率都低于 13%，因为故障只影响了一个 B4 超级节点，大多数流仍经过正常的超级节点。然而，中断造成的破坏比这一丢失率看起来更严重，因为其失效行为呈双峰分布，

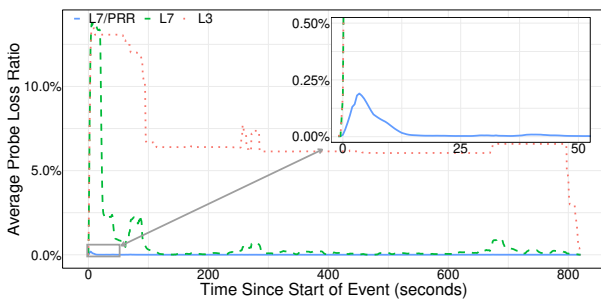
这是非拥塞中断的典型特征：所有经过故障超级节点的流都遭遇 100% 丢包，而经过正常超级节点的流则保持正常的低丢失率。对于使用了某些故障路径的客户来说，即使大多数路径依然正常，网络在没有 PRR 的情况下也实际上不可用了，因为应用会停滞。

L7 曲线展示了开发 PRR 之前的中断恢复行为。L7 丢失率最初与 L3 相同，但在 20s 后大幅下降，此后缓慢衰减，偶尔出现尖峰。这一改善来自 RPC 层，即应用层恢复：在连续 20s 没有取得进展后，RPC 层会打开一条新连接。由于新连接具有不同的端口号，它们经由 ECMP 选择了不同的网络路径，从而绕开中断。大多数新路径碰巧可用，因为 L3 丢失率表明，最初平均只有 13% 的路径失效。这与 PRR 的重新选路类似，但有两点区别：(1) PRR 借助 FlowLabel，无需重建 TCP 连接就能重新选路；(2) PRR 工作在 RTT 时间尺度，远短于 20s 的 RPC 超时。

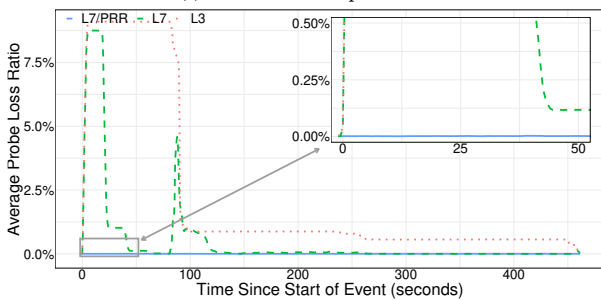
一些 TCP 流需要尝试多次才能找到可用路径，因此连通性恢复过程出现了长尾。后续尖峰来自路由更新：更新改变了路径和 ECMP 映射，导致某些原本正常的连接进入黑洞。在这个过程中，TCP 重传几乎无济于事，因为在某一时刻，一条连接要么正常工作，要么丢失其全部分组。

最后，L7/PRR 曲线使用与 L7 相同的 RPC 探测，但启用了 PRR。丢失率大幅下降，低到只有在插图中才能看清。其修复速度约为 L7 情况的 100 倍，洲内路径尤其如此，因为 RTT 更短。这达到了预期效果：连通性中断很短，不会触发应用层恢复机制，因此大多数客户并未察觉发生了中断。

这个案例揭示了我们更广泛观察到的中断特征：许多中断会使某个区域对之间的部分路径成为黑洞，同时仍有很多路径可用；还有一些中断无法通过快速重路由修复，因此持续很久，如果不能快速恢复，就会干扰用户。兼具这两个特征的中断，为 PRR 提升网络可用性提供了机会。

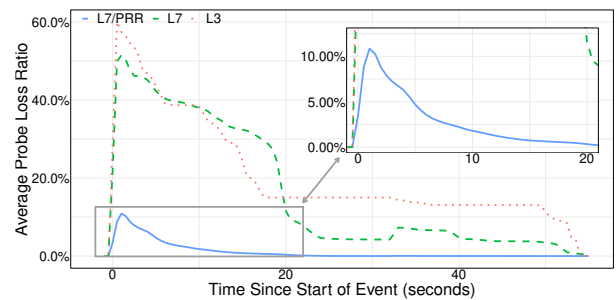


(a) Inter-continental probe loss

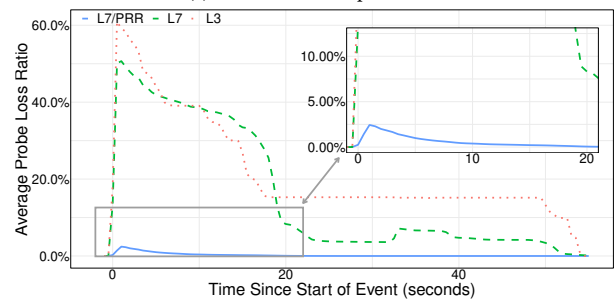


(b) Intra-continental probe loss

图 5：复杂 B4 中断期间的探测丢失情况。



(a) Inter-continental probe loss



(b) Intra-continental probe loss

图 6：B4 光链路故障期间的探测丢失情况。

图中文字对照：两图中，Time Since Start of Event (seconds)：事件发生后经过的时间（秒）；Average Probe Loss Ratio：平均探测丢失率；(a) Inter-continental probe loss：跨洲探测丢失；(b) Intra-continental probe loss：洲内探测丢失。L3、L7、L7/PRR 分别对应正文定义的三种探测，插图为局部放大。

案例 2：光链路故障。接下来考察一次导致 B4 骨干网部分容量丧失的光链路故障。图 6 展示了中断期间，跨洲和洲内路径的探测丢失率随时间的变化。事件刚开始时，L3 丢失率约为 60%，表明大多数路径已经失效，但仍有部分容量可用。

中断开始后，快速路由修复机制立即采取行动，在 5s 内把 L3 丢失率降至约 40%。进一步的改善使丢失率在事件发生后约 20s 降至约 20%。修复缓慢的原因，是大量出方向容量丧失造成绕行链路拥塞，以及同时更新大量路由所带来的 SDN 编程延迟。最终，在 60s 后，流量工程绕开了无响应的数据平面组件，中断才被解决。

L7 曲线起初略低于 L3，因为 L7 探测具有 2s 的超时窗口，在这段时间内路由已修复了一些路径。可以看到，在最初 20s 内，TCP 无法缓解探测丢失，因为重传不能带来超出路由恢复本身的帮助。事实上，约 10s 后 L7 丢失率反而超过了 L3，因为指数退避延迟了对可用路径的发现。约 20s 时，RPC 通道重建使洲内和跨洲路径在剩余中断期间的丢失率大致减半。这些效应都与我们的仿真结果（第 3 节）一致。

相比之下，PRR 降低了探测丢失峰值，并迅速消除了中断影响。对于洲内路径，L7/PRR 将探测丢失峰值降至 2.4%，并在中断开始后 20s 时完全消除了丢失。对于跨洲路径，L7/PRR 同样表现良好，探测丢失峰值约为 11%，比 L3 峰值低 5 倍以上。这个案例说明了路径 RTT 如何影响 PRR。与仿真（第 3 节）一致，RTT 更低的洲内路径具有更低的峰值，并恢复得更快。在两类路径上，PRR 都在 L7 的基础上进一步大幅减少了探测丢失。

案例 3：单台设备的线卡问题。下一个中断涉及 B2 骨干网中的一台设备（图 7）。中断期间，该设备的两块线卡发生故障，导致部分跨洲路径出现探测丢失。由于这种故障的性质，路由没有作出响应。最终，一个自动化过程将负载从该设备上排空，并将其退出服务，才缓解了中断。

虽然这次中断的原因与前述案例不同，但结果相似：PRR 能够大幅减少丢失。L3 的探测丢失峰值为 19%；L7/PRR 将其降至 1.2%，降低超过 15 倍，而且与前一个中断一样，在 20s 后迅速将丢失水平降到接近零。相反，L7 探测的丢失峰值高达 14%，持续时间也明显长于 L7/PRR。

案例 4：区域光纤切断。最后，我们介绍一次给 PRR 带来挑战的中断（图 8）。在这次事件中，光纤切断造成了大量容量丧失。平均 L3 探测丢失率峰值达到 70%，并在约 50% 或更高水平上持续了 3 分钟。这个严重故障影响了很多路径。快速重路由没能缓解问题，因为容量大幅下降使绕行路径过载。约 3 分钟后，全局路由将流量从中断区域移开，降低了丢失率，并缓解了绕行路径的拥塞。

L7/PRR 将峰值丢失率降到 14%，改善了 5 倍。相比之下，L7 仅将峰值降至 65%，因此 L7/PRR 要有效得多；但由于失效路径比例很高，它无法完全修复这次中断。事件期间的路由更新进一步加重了路径失效：PRR 把连接移到可用路径后，ECMP 映射变化又使其中一些连接回到了失效路径。因此，可以看到 L7/PRR 丢失率随时间下降，但不断被一系列尖峰打断。

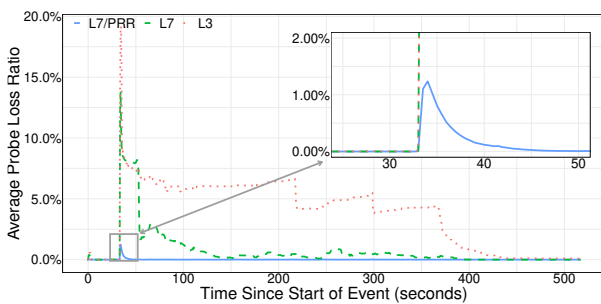


图 7：B2 设备故障期间的跨洲探测丢失情况。未观察到洲内探测丢失。

图中文字对照：Time Since Start of Event (seconds)：事件发生后经过的时间（秒）；Average Probe Loss Ratio：平均探测丢失率；曲线标签 L3、L7、L7/PRR 的含义同正文。

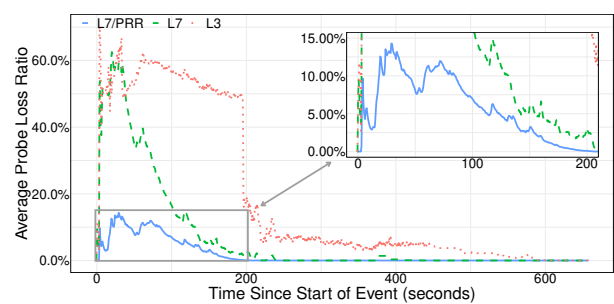


图 8：B2 区域光纤切断期间的洲内探测丢失情况。跨洲曲线与之相似，因此省略。

4.3 总体改善

案例研究详细展示了 PRR 如何修复单次中断。为了量化 PRR 对整体网络可用性的提升，我们汇总了六个月研究期间，Google 网络所有区域对上全部中断的测量结果。总中断时间中的绝大多数来自短暂或小规模中断。结果表明，PRR 对这些中断同样有效，也能应对长时间、大规模中断。

中断分钟数。我们的目标是提高可用性，其定义为

$$\frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}},$$

其中 MTBF 为平均故障间隔时间，MTTR 为平均修复时间。该公式等价于 1 减去中断时间所占的比例。由于保密要求，我们无法报告绝对可用性指标，因此报告 L3、L7 和 L7/PRR 各层之间中断时间的相对减少量。这些相对减少量可以直接换算成可用性收益。例如，中断时间减少 90% 相当于为可用性增加一个“9”，譬如假设从 99% 提高到 99.9%。

我们根据流级别的探测丢失，以分钟为单位测量 L3、L7 和 L7/PRR 的中断时间。具体做法是计算每条流在每分钟内的探测丢失率。如果某条流的丢失率超过 5%，即高于正常情况下可接受的较低丢失水平，就将其标记为有损流。如果一对网络区域在某个 1 分钟区间内，有损流比例超过 5%，表明问题并非局限于个别流，就将该区间记为这个区域对的一个中断分钟。我们还会进一步缩减计时范围，只保留其中存在探测丢失的 10s 区间，以免对于在某分钟内才开始或已经结束的中断仍计入整整一分钟。

图 9 给出了 L7/PRR 探测相对于 L3 探测，修复了多少比例的总中断分钟数。我们分别报告 B2 和 B4 两个骨干网的结果，并进一步区分洲内和跨洲路径。同样，为理解收益来源，我们还比较了未启用 PRR 的 L7 与 L3，以及 L7/PRR 与 L7。计算覆盖数千个区域对，包含数百次中断事件。

相对于 **L3**，**PRR** 使中断分钟数减少 **64–87%**。PRR 与传输层、应用层恢复机制结合后，能够非常有效地缩短应用所感知的 IP 网络中断。在两个骨干网中，使用 L7/PRR 都显著减少了中断分钟数。减少幅度从 B4 跨洲路径的 64%，到 B2 洲内路径的 87% 不等。注意，与单次中断不同，在跨多次中断汇总时，我们没有观察到洲内与跨洲结果之间的一致模式。这是因为 PRR 的效果不仅取决于 RTT，也取决于拓扑因素。

译者注：原文此处写作“inter- and inter-continental”，重复了 inter。结合本段及图 9 的比较维度，译为“洲内与跨洲”，并在此标明这一文字疑点。

相对于 **L7**，**PRR** 使中断分钟数减少 **54–78%**。对于 L7 探测中未被 TCP 和应用层恢复机制修复的中断分钟，PRR 能修复其中的大部分。这证实了 L7/PRR 相对 L3 的大部分改善，并不来自 RPC 层或 TCP 重传：L7 相对 L3 仅使累计中断分钟数减少了 15–42%。我们认为，这部分收益主要来自 RPC 重连，因为 TCP 重传对黑洞无效。

译者注：本节相对 L7 的区间为 54–78%，而摘要、引言和结论中的 RPC 流量收益区间为 63–84%。用户提供的正式出版 PDF 中，两种数值都存在；本译本分别保留，没有自行统一数值或补造解释。

PRR 的效果在时间上保持良好。作为一项核查，我们也考察了 PRR 行为随时间的变化。图 10 展示了每日中断分钟修复比例经过广义加性模型 (GAM) [43] 平滑后的结果。可以看到一定的时间变化，反映了中断性质的变化；但在整个研究期间，PRR 始终带来了较大幅度的中断分钟数减少。

4.4 对不同区域对的效果

中断会影响多个区域对，而各个区域对可能呈现不同的修复行为。接下来，我们考察 PRR 收益在网络中不同区域对之间的分布。图 11 给出了不同协议层次之间，中断分钟修复比例在区域对上的

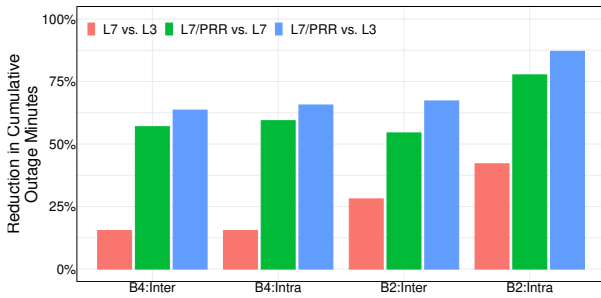


图 9: B2 和 B4 骨干网上, 洲内与跨洲路径的中断分钟数减少比例。

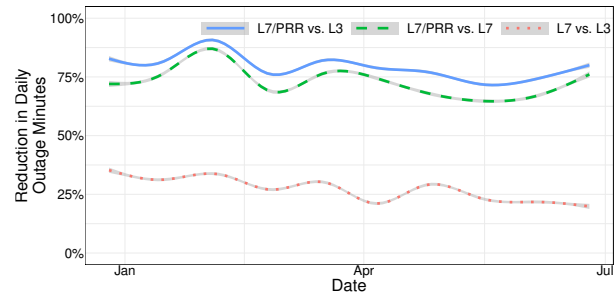


图 10: 中断分钟数减少比例随时间的变化。

图中文字对照: 图 9: Reduction in Cumulative Outage Minutes, 累计中断分钟数减少比例; Inter / Intra, 跨洲 / 洲内。图 10: Reduction in Daily Outage Minutes, 每日中断分钟数减少比例; Date, 日期; Jan / Apr / Jul, 1 月 / 4 月 / 7 月。两图中的“X vs. Y”表示 X 相对于 Y 的比较。

互补累积分布函数 (CCDF)。图中覆盖整个研究期间全网的所有区域对。点的位置越高、越靠右, 结果越好, 因为这意味着更多区域对修复了更大比例的中断分钟数。

PRR 对多种路径都表现良好。可以看到, 在两个骨干网中, 绝大多数区域对使用 L7/PRR 后都比 L3 获得了显著收益。对于 B2 的洲内和跨洲区域对, 分别有 50% 和 16% 能够修复 100% 的中断分钟。B4 上 PRR 的表现更分散: 洲内和跨洲区域对中, 分别有 63% 和 77% 的区域对, 中断分钟数减少了一半。

PRR 相对于 **L7** 有显著改善。如预期所示, L7/PRR 的收益远大于 L7。不论以 L3 还是 L7 为基线, 表示 PRR 收益的曲线都相当接近, 而且几乎所有区域对的中断分钟数都减少了。例外通常是那些中断分钟本来就很少、且 L7/PRR 动态过程在采样时碰巧不利的区域对。相反, 对于 3-16% 的区域对, 不带 PRR 的 L7 相对 L3 反而增加了中断分钟数。这种看似反直觉的结果之所以可能出现, 是因为 TCP 在失效路径上的指数退避往往会延长中断时间, 直到触及 RPC 超时。

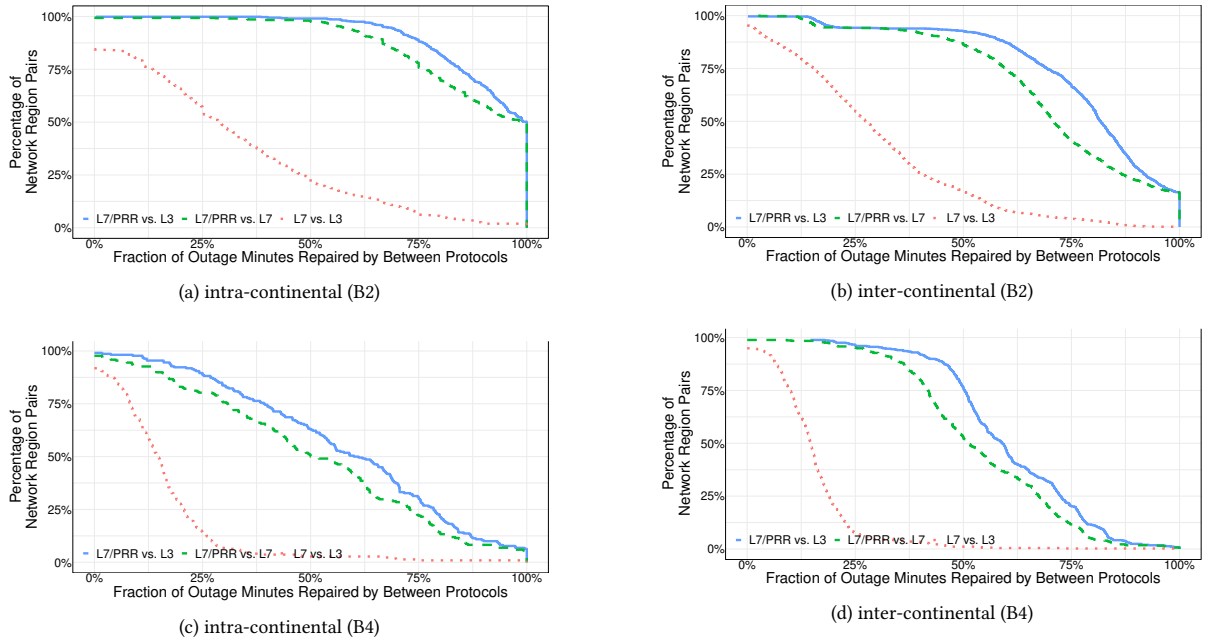


图 11: 不同区域对上改善程度的 CCDF。

图中文字对照：横轴：协议间比较得到的中断分钟修复比例；纵轴：网络区域对百分比。子图依次为 (a) B2 洲内、(b) B2 跨洲、(c) B4 洲内、(d) B4 跨洲。L7/PRR vs. L3、L7/PRR vs. L7、L7 vs. L3 分别表示相应协议层次间的比较。为便于单栏阅读，原图纵向排列的四个子图改为两行两列，数据与曲线未改动。

5 讨论

原文位置：PDF 第 9–10 页／原刊第 692–693 页。

下面简要讨论 PRR 的一些其他方面。

其他传输协议。PRR 可以用于保护所有传输协议，包括多路径传输协议，因为所有可靠传输协议都会检测交付失败。例如，我们也将 PRR 用于 Pony Express [31] 的绕过操作系统的流量，其做法与 TCP 只有细微差异。用户态 UDP 传输协议可以在检测到网络问题时，通过系统调用改变 FlowLabel，从而实现重新选路。即使 DNS 和 SNMP 这样的协议，也可以在重试时改变 FlowLabel 来提高可靠性。

Cloud 与封装。由于虚拟化会影响 ECMP，必须扩展 PRR 才能保护 Cloud 客户的 IPv6 流量。Google Cloud 的虚拟化机制 [12] 使用 PSP 加密 [19]，在原始虚拟机 (VM) 分组外添加 IP / UDP / PSP 首部，如图 12 所示。在网络中，交换机使用外层首部进行 ECMP，而忽略 VM 分组的首部。为了让 VM 能通过 FlowLabel 重新选路，我们对 VM 首部进行哈希，并将结果用于外层首部。

这样，当客户机操作系统具备 PRR，且某条 TCP 连接检测到中断并改变其 FlowLabel 时，封装首部也会随之改变，进而使 ECMP 为该连接重新选路。对于不同的封装格式（例如 IPSEC、外层使用 IPv4 等），具体细节会有所差异，但这种传递方式是相同的。

我们也利用封装为 Cloud IPv4 流量启用 PRR。在这种情况下，VM 分组没有 FlowLabel，因此 gve 驱动 [20] 会将用于传递路径选择信号的元数据交给虚拟机监控器 (hypervisor)，后者对其进行哈希，并将结果用于封装首部。

部署。为了保护现有部署，向后兼容和渐进式部署非常重要。PRR 在这方面表现突出。由于需要厂商参与、上游内核修改和交换机升级，推广过程耗时较长，但除此之外相当直接。主机可以

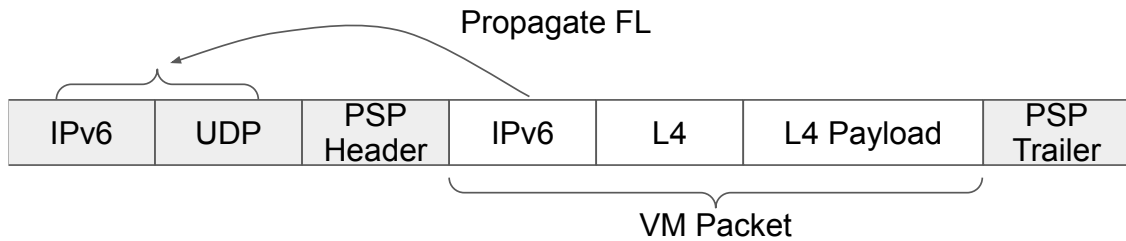


图 12: 通过 PSP 封装为虚拟机提供 PRR。

图中文字对照: Propagate FL: 传递 FlowLabel 所携带的路径选择信息; PSP Header: PSP 首部; PSP Trailer: PSP 尾部; VM Packet: 虚拟机分组; L4: 传输层首部; L4 Payload: 传输层载荷。IPv6、UDP、PSP 等协议名称保持原文。

按任意顺序升级, 为发出的 TCP 流量使用 FlowLabel。与此同时, 交换机也可以按任意顺序升级, 将 FlowLabel 纳入 ECMP 哈希; 按照 FlowLabel 的语义, 这种改变并无害处。

PRR 要起作用, 并不需要所有交换机都对 FlowLabel 进行哈希, 只需故障上游的某些交换机这样做。通常, 只升级一部分交换机, 就能获得相当程度的保护。这一性质对全球互联网中 IPv6 流量的可靠性具有意义。各个提供商不仅可以启用 FlowLabel 哈希来保护自身网络, 上游提供商还能够通过改变进入下游网络的入口, 在一定程度上绕开下游故障。

6 相关工作

原文位置: PDF 第 10–11 页 / 原刊第 693–694 页。

大多数提高网络可靠性的工作关注网络内部, 例如快速重路由 [3, 4]、骨干网 [23, 40, 47] 或数据中心 [44]。报告中断成因 [17, 21, 41], 或讨论适合这些中断的可用性指标的论文相对较少; 后者的一个例子是窗口化可用性 [22], 它区分短时间与长时间中断。

主机有潜力利用 FlowLabel 提升可用性 [5], 但据我们所知, 此前尚未有大规模部署或评估。大多数主机侧工作聚焦于 MPTCP [6]、SRD [38] 和多路径 QUIC [13] 等传输协议, 它们在一组网络路径上传输消息。虽然主要目的是提升性能, 这些协议也能提高可用性。例如, MPTCP 可以在发生 RTO 时, 将一个子流中的数据改由另一个子流传输。但它们只使用一小组路径, 而且可能无法保护连接建立阶段的可靠性; 例如, MPTCP 只有在三次握手成功后才添加路径 [5]。将 PRR 加入多路径传输协议后, 可以通过不断探索新路径、直到找到可用路径来提高可靠性, 并保护连接建立过程。

另一类相关主机行为已有大量研究, 即为负载均衡而使用多路径 [1, 6, 14–16, 27–30, 32, 37, 39, 42, 45, 46]。其中大多数工作并不是在 IPv6 背景下进行的, 也没有考虑 FlowLabel; 而我们发现, FlowLabel 很适合多路径。在我们的网络中, PRR 和 PLB [32] 被一起实现, 用重新选路同时完成负载均衡与绕开故障。这些工作中也有一些考虑了可靠性, 例如 CLOVE [28] 利用 ECMP 探明可用路径。PRR 表明这种映射并非必需, 因为随机抽取路径就能取得良好效果。

最后, 文献 [7] 主张, 在路径选择中, 主机应当发挥更大的作用, 而不只是由路由器响应故障。PRR 是这一主张的一种实现。

7 结论

原文位置: PDF 第 11 页 / 原刊第 694 页。

PRR 已在 Google 全网部署, 并已保护几乎全部生产流量的可靠性数年之久。我们的 Cloud 客户也可以使用它。PRR 大幅缩短了用户可感知的中断。在一项覆盖两个骨干网、持续 6 个月的研究中, 它使 RPC 流量的区域对累计中断时长减少了 63–84%。这相当于为可用性增加了 0.4–0.8 个“9”。现在, 我们要求所有传输协议都使用 PRR。

PRR 及其姊妹技术 PLB [32], 体现了我们网络架构的一次转变。在早期互联网 [11] 中, 网络会指示主机如何行动, 例如 ICMP Source Quench (源抑制) 消息。但这种方式效果不佳。在现代互联网中, 主机和路由器互不发出指令: 主机发送分组, 路由器决定如何处理它们。

在我们的架构中, 主机使用 IPv6 FlowLabel 来指示网络如何为其流量选择路径。这种转变源于网络通过增加并行链路来扩展容量, 从而极大地增加了路径多样性。为了充分利用这种多样性, 我们依靠路由让主机能够使用众多路径, 并由主机在这些路径之间转移流量, 以提高可靠性与性能。我们希望这种由 FlowLabel 支持的架构方法能够得到广泛采用。

致谢

许多同事帮助我们实现了 PRR。我们尤其感谢 Piotr Marecki、Dan Ameme、Jay Sanghera、Yousuk Seung、Chang Liu、Arda Balkanay、Yuval Ehrental、Przemyslaw Krol、James Spooner、Mike Chen 和 Steve Plunkett, 感谢他们在 PRR 开发与部署方面的帮助。Alex Snoeren 和 Neal Cardwell 不知疲倦地与我们合作, 使本文和 PRR 得以尽可能完善。Yaogong Wang、Darin Peetz、Willem de Bruijn、Brian Gardiner 和 Yong Xia 帮助我们为 Google Cloud 提供了 PRR 支持。Maxwell Chen 帮助收集了测量数据。

参考文献

- [1] Mohammad Alizadeh, Tom Edsall, Sarang Dharmapurikar, Ramanan Vaidyanathan, Kevin Chu, Andy Fingerhut, Vinh The Lam, Francis Matus, Rong Pan, Navindra Yadav, and George Varghese. 2014. CONGA: Distributed Congestion-Aware Load Balancing for Datacenters. In Proceedings of the 2014 ACM Conference on SIGCOMM (SIGCOMM '14). Association for Computing Machinery, New York, NY, USA, 503–514. <https://doi.org/10.1145/2619239.2626316>
- [2] Shane Amante, Jarno Rajahalme, Brian E. Carpenter, and Sheng Jiang. 2011. IPv6 Flow Label Specification. RFC 6437. (Nov. 2011). <https://doi.org/10.17487/RFC6437>
- [3] Alia Atlas, George Swallow, and Ping Pan. 2005. Fast Reroute Extensions to RSVP-TE for LSP Tunnels. RFC 4090. (May 2005). <https://doi.org/10.17487/RFC4090>
- [4] Alia Atlas and Alex D. Zinin. 2008. Basic Specification for IP Fast Reroute: Loop-Free Alternates. RFC 5286. (Sept. 2008). <https://doi.org/10.17487/RFC5286>
- [5] Alexander Azimov. 2020. Self-healing Network or The Magic of Flow Label. <https://ripe82.ripe.net/presentations/20-azimov.ripe82.pdf>. (2020).
- [6] Olivier Bonaventure, Christoph Paasch, and Gregory Detal. 2017. Use Cases and Operational Experience with Multipath TCP. RFC 8041. (Jan. 2017). <https://doi.org/10.17487/RFC8041>
- [7] Matthew Caesar, Martin Casado, Teemu Koponen, Jennifer Rexford, and Scott Shenker. 2010. Dynamic Route Recomputation Considered Harmful. ACM SIGCOMM Computer Communication Review 40, 2 (Apr. 2010), 66–71. <https://doi.org/10.1145/1764873.1764885>
- [8] Neal Cardwell, Yuchung Cheng, and Eric Dumazet. 2016. TCP Options for Low Latency: Maximum ACK Delay and Microsecond Timestamps, IETF 97 tcpm. <https://datatracker.ietf.org/meeting/97/materials/slides-97-tcpm-tcp-options-for-low-latency-00>. (2016).
- [9] Sid Chaudhuri, Gisli Hjalmtysson, and Jennifer Yates. 2000. Control of lightpaths in an optical network. In Optical Internetworking Forum.
- [10] Yuchung Cheng, Neal Cardwell, Nandita Dukkkipati, and Priyaranjan Jha. 2021. The RACK-TLP Loss Detection Algorithm for TCP. RFC 8985. (Feb. 2021). <https://doi.org/10.17487/RFC8985>
- [11] David Clark. 1988. The Design Philosophy of the DARPA Internet Protocols. SIGCOMM Comput. Commun. Rev. 18, 4 (aug 1988), 106–114. <https://doi.org/10.1145/52325.52336>
- [12] Mike Dalton, David Schultz, Ahsan Arefin, Alex Docauer, Anshuman Gupta, Brian Matthew Fahs, Dima Rubinstein, Enrique Cauich Zermeno, Erik Rubow, Jake Adriaens, Jesse L Alpert, Jing Ai, Jon Olson, Kevin P. DeCaboooter, Marc Asher de Kruijf, Nan Hua, Nathan Lewis, Nikhil Kasinadhuni, Riccardo Crepaldi, Srinivas Krishnan, Subbaiah Venkata, Yossi Richter, Uday Naik, and Amin Vahdat. 2018. Andromeda: Performance, Isolation, and Velocity at Scale in Cloud Network Virtualization. In 15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2018. USENIX Association, Renton, WA, 373–387.
- [13] Quentin De Coninck and Olivier Bonaventure. 2017. Multipath QUIC: Design and Evaluation. In Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies.
- [14] Advait Dixit, Pawan Prakash, Y. Charlie Hu, and Ramana Rao Kompella. 2013. On the impact of packet spraying in data center networks. In 2013 Proceedings IEEE INFOCOM. 2130–2138. <https://doi.org/10.1109/INFOCOM.2013.6567015>

- [15] Yilong Geng, Vimalkumar Jeyakumar, Abdul Kabbani, and Mohammad Alizadeh. 2016. Juggler: A Practical Reordering Resilient Network Stack for Datacenters. In Proceedings of the Eleventh European Conference on Computer Systems (EuroSys '16). Association for Computing Machinery, New York, NY, USA, Article 20, 16 pages. <https://doi.org/10.1145/2901318.2901334>
- [16] Soudeh Ghorbani, Zibin Yang, P. Brighten Godfrey, Yashar Ganjali, and Amin Firoozshahian. 2017. DRILL: Micro Load Balancing for Low-Latency Data Center Networks. In Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '17). Association for Computing Machinery, New York, NY, USA, 225–238. <https://doi.org/10.1145/3098822.3098839>
- [17] Phillipa Gill, Navendu Jain, and Nachiappan Nagappan. 2011. Understanding Network Failures in Data Centers: Measurement, Analysis, and Implications. SIGCOMM Comput. Commun. Rev. 41, 4 (aug 2011), 350–361.
- [18] Google. 2015. gRPC Motivation and Design Principles (2015-09-08). <https://grpc.io/blog/principles/>. (2015).
- [19] Google. 2022. PSP Architecture Specification (2022-11-17). https://github.com/google/psp/blob/main/doc/PSP_Arch_Spec.pdf. (2022).
- [20] Google. 2022. Using Google Virtual NIC. <https://cloud.google.com/compute/docs/networking/using-gvnic>. (2022).
- [21] Ramesh Govindan, Ina Minei, Mahesh Kallahalla, Bikash Koley, and Amin Vahdat. 2016. Evolve or Die: High-Availability Design Principles Drawn from Googles Network Infrastructure. In Proceedings of the 2016 ACM SIGCOMM Conference (SIGCOMM '16). Association for Computing Machinery, New York, NY, USA, 58–72. <https://doi.org/10.1145/2934872.2934891>
- [22] Tamás Hauer, Philipp Hoffmann, John Lunney, Dan Ardelean, and Amer Diwan. 2020. Meaningful Availability. In 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20). USENIX Association, Santa Clara, CA, 545–557. <https://www.usenix.org/conference/nsdi20/presentation/hauer>
- [23] Chi-Yao Hong, Subhasree Mandal, Mohammad Al-Fares, Min Zhu, Richard Alimi, Kondapa Naidu B., Chandan Bhagat, Sourabh Jain, Jay Kaimal, Shiyu Liang, Kirill Mendelev, Steve Padgett, Faro Rabe, Saikat Ray, Malveeka Tewari, Matt Tierney, Monika Zahn, Jonathan Zolla, Joon Ong, and Amin Vahdat. 2018. B4 and after: Managing Hierarchy, Partitioning, and Asymmetry for Availability and Scale in Google's Software-Defined WAN. In Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18). Association for Computing Machinery, New York, NY, USA, 74–87. <https://doi.org/10.1145/3230543.3230545>
- [24] Christian Hopps and Dave Thaler. 2000. Multipath Issues in Unicast and Multicast Next-Hop Selection. RFC 2991. (Nov. 2000). <https://doi.org/10.17487/RFC2991>
- [25] Van Jacobson. 1988. Congestion Avoidance and Control. SIGCOMM Comput. Commun. Rev. 18, 4 (aug 1988), 314–329. <https://doi.org/10.1145/52325.52356>
- [26] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, Jon Zolla, Urs Hölzle, Stephen Stuart, and Amin Vahdat. 2013. B4: Experience with a Globally-Deployed Software Defined Wan. SIGCOMM Comput. Commun. Rev. 43, 4 (aug 2013), 3–14.

- [27] Abdul Kabbani, Balajee Vamanan, Jahangir Hasan, and Fabien Duchene. 2014. FlowBender: Flow-Level Adaptive Routing for Improved Latency and Throughput in Datacenter Networks. In Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies (CoNEXT '14). Association for Computing Machinery, New York, NY, USA, 149–160. <https://doi.org/10.1145/2674005.2674985>
- [28] Naga Katta, Aditi Ghag, Mukesh Hira, Isaac Keslassy, Aran Bergman, Changhoon Kim, and Jennifer Rexford. 2017. Clove: Congestion-Aware Load Balancing at the Virtual Edge. In Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '17). Association for Computing Machinery, New York, NY, USA, 323–335. <https://doi.org/10.1145/3143361.3143401>
- [29] Naga Katta, Mukesh Hira, Changhoon Kim, Anirudh Sivaraman, and Jennifer Rexford. 2016. HULA: Scalable Load Balancing Using Programmable Data Planes. In Proceedings of the Symposium on SDN Research (SOSR '16). Association for Computing Machinery, New York, NY, USA, Article 10, 12 pages. <https://doi.org/10.1145/2890955.2890968>
- [30] Ming Li, Deepak Ganesan, and Prashant Shenoy. 2009. PRESTO: Feedback-Driven Data Management in Sensor Networks. IEEE/ACM Transactions on Networking 17, 4 (2009), 1256–1269. <https://doi.org/10.1109/TNET.2008.2006818>
- [31] Michael Marty, Marc de Kruijf, Jacob Adriaens, Christopher Alfeld, Sean Bauer, Carlo Contavalli, Mike Dalton, Nandita Dukkupati, William C. Evans, Steve Gribble, Nicholas Kidd, Roman Kononov, Gautam Kumar, Carl Mauer, Emily Musick, Lena Olson, Mike Ryan, Erik Rubow, Kevin Springborn, Paul Turner, Valas Valancius, Xi Wang, and Amin Vahdat. 2019. Snap: a Microkernel Approach to Host Networking. In In ACM SIGOPS 27th Symposium on Operating Systems Principles. New York, NY, USA.
- [32] Mubashir Adnan Qureshi, Yuchung Cheng, Qianwen Yin, Qiaobin Fu, Gautam Kumar, Masoud Moshref, Junhua Yan, Van Jacobson, David Wetherall, and Abdul Kabbani. 2022. PLB: Congestion Signals Are Simple and Effective for Network Load Balancing. In Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM '22). Association for Computing Machinery, New York, NY, USA, 207–218. <https://doi.org/10.1145/3544216.3544226>
- [33] Costin Raiciu, Sebastien Barre, Christopher Pluntke, Adam Greenhalgh, Damon Wischik, and Mark Handley. 2011. Improving Datacenter Performance and Robustness with Multipath TCP. SIGCOMM Comput. Commun. Rev. 41, 4 (aug 2011), 266–277.
- [34] Jarno Rajahalme, Alex Conta, Brian E. Carpenter, and Dr. Steve E Deering. 2004. IPv6 Flow Label Specification. RFC 3697. (March 2004). <https://doi.org/10.17487/RFC3697>
- [35] Matt Sargent, Jerry Chu, Dr. Vern Paxson, and Mark Allman. 2011. Computing TCP's Retransmission Timer. RFC 6298. (June 2011). <https://doi.org/10.17487/RFC6298>
- [36] Pasi Sarolahti and Alexey Kuznetsov. 2002. Congestion Control in Linux TCP. In 2002 USENIX Annual Technical Conference (USENIX ATC 02). USENIX Association, Monterey, CA. <https://www.usenix.org/conference/2002-usenix-annual-technical-conference/congestion-control-linux-tcp>
- [37] Siddhartha Sen, David Shue, Sunghwan Ihm, and Michael J. Freedman. 2013. Scalable, Optimal Flow Routing in Datacenters via Local Link Balancing. In Proceedings of the Ninth ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT '13). Association for Computing Machinery, New York, NY, USA, 151–162. <https://doi.org/10.1145/2535372.2535397>
- [38] Leah Shalev, Hani Ayoub, Nafea Bshara, and Erez Sabbag. 2020. A Cloud-Optimized Transport Protocol for Elastic and Scalable HPC. IEEE Micro 40, 6 (2020), 67–73. <https://doi.org/10.1109/MM.2020.3016891>

- [39] Shan Sinha, Srikanth Kandula, and Dina Katabi. 2004. Harnessing TCP's burstiness with flowlet switching. In Proc. 3rd ACM Workshop on Hot Topics in Networks (Hotnets-III).
- [40] Sucha Supittayapornpong, Barath Raghavan, and Ramesh Govindan. 2019. Towards Highly Available Clos-Based WAN Routers. In Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM '19). Association for Computing Machinery, New York, NY, USA, 424–440. <https://doi.org/10.1145/3341302.3342086>
- [41] Daniel Turner, Kirill Levchenko, Alex C. Snoeren, and Stefan Savage. 2010. California Fault Lines: Understanding the Causes and Impact of Network Failures. SIGCOMM Comput. Commun. Rev. 40, 4 (aug 2010), 315–326. <https://doi.org/10.1145/1851275.1851220>
- [42] Erico Vanini, Rong Pan, Mohammad Alizadeh, Parvin Taheri, and Tom Edsall. 2017. Let It Flow: Resilient Asymmetric Load Balancing with Flowlet Switching. In 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17). USENIX Association, Boston, MA, 407–420. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/vanini>
- [43] Simon N Wood. 2017. Generalized additive models: an introduction with R (second ed.). Chapman and Hall/CRC, Boca Raton. <https://doi.org/10.1201/9781315370279>
- [44] Dingming Wu, Yiting Xia, Xiaoye Steven Sun, Xin Sunny Huang, Simbarashe Dzinamarira, and T. S. Eugene Ng. 2018. Masking Failures from Application Performance in Data Center Networks with Shareable Backup. In Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18). Association for Computing Machinery, New York, NY, USA, 176–190. <https://doi.org/10.1145/3230543.3230577>
- [45] David Zats, Tathagata Das, Prashanth Mohan, Dhruba Borthakur, and Randy Katz. 2012. DeTail: Reducing the Flow Completion Time Tail in Datacenter Networks. In Proceedings of the ACM SIGCOMM 2012 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '12). Association for Computing Machinery, New York, NY, USA, 139–150. <https://doi.org/10.1145/2342356.2342390>
- [46] Hong Zhang, Junxue Zhang, Wei Bai, Kai Chen, and Mosharaf Chowdhury. 2017. Resilient Datacenter Load Balancing in the Wild. In Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '17). Association for Computing Machinery, New York, NY, USA, 253–266. <https://doi.org/10.1145/3098822.3098841>
- [47] Zhizhen Zhong, Manya Ghobadi, Alaa Khaddaj, Jonathan Leach, Yiting Xia, and Ying Zhang. 2021. ARROW: Restoration-Aware Traffic Engineering. In Proceedings of the 2021 ACM SIGCOMM 2021 Conference (SIGCOMM '21). Association for Computing Machinery, New York, NY, USA, 560–579. <https://doi.org/10.1145/3452296.3472921>
- [48] Junlan Zhou, Malveeka Tewari, Min Zhu, Abdul Kabbani, Leon Poutievski, Arjun Singh, and Amin Vahdat. 2014. WCMP: Weighted Cost Multipathing for Improved Fairness in Data Centers. Article No. 5. <https://dl.acm.org/doi/10.1145/2592798.2592803>